AI Solution Engineer Technical Assessment

Akbar Jaya Pratama Putra

V-TEKI · 12 November 2026

1. Summary
2. Case 1 - Fraud Risk Prediction
3. Case 2 - Document Intelligence
4. Case 3 - Enterprise Knowledge Chatbot
5. Critical Thinking & Engineering Judgment
6. AI tool use

Summary

Three working prototypes, each with synthetic data, automated tests, an evaluation with real results, and privacy controls. Everything runs on a laptop; no data is sent to external services.

Case	What it does	Key result	Tests
1. Fraud Risk Prediction	Ranks transactions so analysts review the riskiest first	Catches 95% of fraud with 10 reviews a day	23
2. Document Intelligence	Turns invoice PDFs into validated JSON; unsure results go to a person	95% field accuracy, 0 wrong results auto-accepted	44
3. Knowledge Chatbot	Answers employee questions from SOPs with sources, by role	15/15 correct answers, 0 restricted data leaked	64

Principles used in all three cases

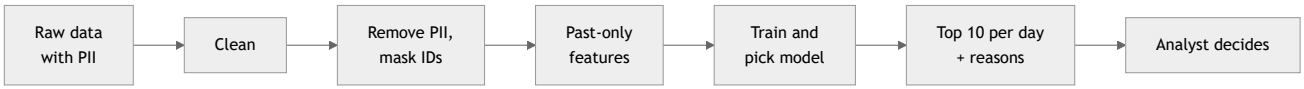
- **People decide, the system assists:** fraud cases and unsure invoices go to a human; the chatbot refuses rather than guesses.
- **Every number is measured**, on a documented evaluation set, and failures are reported.
- **Privacy by design:** only needed data is used, identities are masked, logs hold no content, and AI models run locally.

How to run: each case folder has a README with setup, run and test commands (Python 3.14; Ollama for the AI parts). This report keeps only the essentials; full details are in those READMEs.

Case 1: Fraud Risk Prediction

Problem and solution

- A payments team can review only **10 transactions a day**, so the system **ranks** risk; it never labels anyone as fraud.
- **Data:** synthetic customers, transactions and logins (2,000 / 44,009 / 45,320 rows, 1.3% fraud) with planted problems: mixed ID formats, 862 duplicates, 460 missing timestamps. **Label = confirmed fraud**, not "blocked": 28 blocked customers had no fraud and 30 fraud victims were never blocked.
- **10 features from past events only** (amount vs. 30-day average, transactions in the last hour, failed logins, new device, ...). **Split by time:** Feb-Apr train, May choose the model, June final test.
- **Models:** simple rules vs. logistic regression vs. gradient boosting. Output: risk score, daily rank, reason codes, model version and status `PENDING_REVIEW` for the analyst.



Results (June test set, 84 fraud cases)

Model	PR-AUC	Fraud caught	Precision	Accuracy
Always "not fraud"	0.01	0%	0%	**98.8%**
Rules (baseline)	0.88	95%	27%	96.8%
Logistic regression	0.96	96%	27%	96.8%
Gradient boosting (chosen)	**0.98**	95%	27%	96.8%

- **Accuracy is misleading:** the model that never flags anything has the highest accuracy.
- **Precision 27% is near the ceiling:** 84 frauds but 300 review slots in June, so the maximum is 28%.
- **The ranking works:** 43% of rank 1-3 cases are fraud vs. 3% at rank 10.

Privacy and security

- Names, emails and phones are dropped before modelling; customer and device IDs are replaced with keyed hashes (key kept outside the code).
- Logs mask emails and phones; broken input or a missing key stops the run without output.
- **23 tests**, including a **leakage test** (future data never changes past features) and 6 security tests.

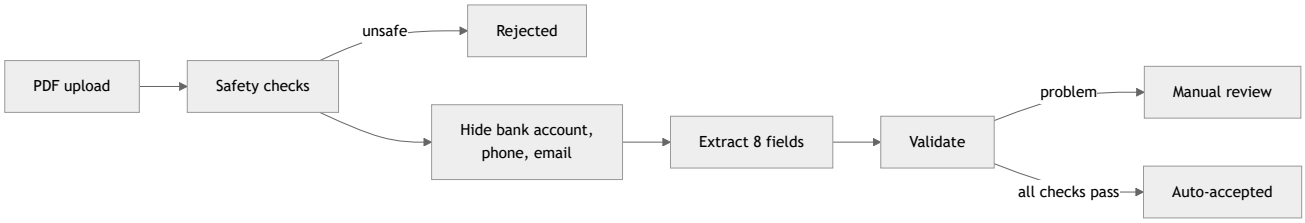
Cost and limitations

- A small daily batch job (≈ US$10-20/month); the real cost is analyst time.
- Synthetic fraud patterns are clearer than real ones, so real performance will be lower.
- Reason codes are simple rules, not an exact explanation of the model.

Case 2: Document Intelligence (Invoices)

Problem and solution

- Extract **8 fields** (number, dates, vendor, tax ID, subtotal, tax, total) from invoice PDFs into JSON.
- **30 synthetic invoices in 3 layouts.** The regex was written from layouts A and B only; layout C (Indonesian labels) plays a **new vendor format**. Some invoices miss fields or have a wrong total on purpose.
- **Two methods compared:** regex vs. a **local LLM** (qwen3 4B in Ollama, JSON output, temperature 0).
- **Validation** checks dates, tax ID format, subtotal + tax = total, 11% VAT, and that every value appears in the document. Results are `AUTO_ACCEPTED` or `NEEDS_REVIEW` with reasons; **no fake confidence percentage**.
- Use it from the command line or an upload API protected by an API key.



Results (30 invoices)

Metric	Regex	Local LLM (chosen)
Field accuracy	67.9%	**95.0%**
Accuracy on the new layout C	4%	**96%**
Missing-field rate	33.2%	3.0%
Auto-accepted	13	16
Auto-accepted but wrong	**0**	**0**
Time per invoice	0.002 s	3.9 s

- Regex is perfect on known layouts but **breaks on a new one**; the LLM adapts.
- **Validation caught every error**, including LLM copy mistakes such as a tax ID with one wrong digit.
- When the LLM got stuck on one invoice, that invoice went to review instead of stopping the batch.

Privacy and security

- The LLM runs locally and receives text with bank accounts, phones and emails removed; `--show-prompt` shows exactly what it gets.
- External AI endpoints are blocked unless approved. Unsafe files (fake PDFs, corrupt, empty, oversized) are rejected.
- The API checks the key and size before reading uploads; logs keep only a file hash. **44 tests.**

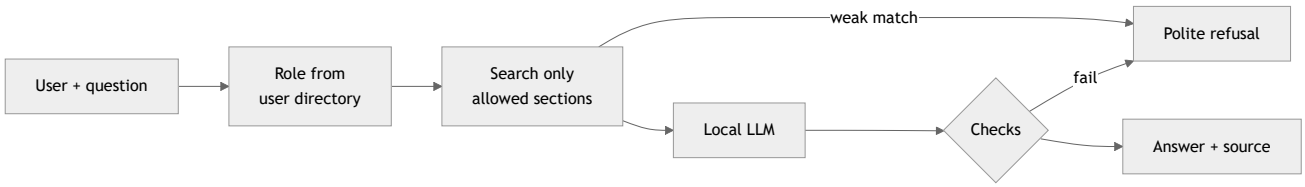
Cost and limitations

- About **IDR 0.16 per invoice** in electricity; an external API would be about US$0.00014 (not used).
- Synthetic invoices are cleaner than real ones; 30 invoices is a small test set; no OCR for scanned invoices.

Case 3: Enterprise Knowledge Chatbot (RAG)

Problem and solution

- Employees ask questions about **8 synthetic SOPs**; 2 are restricted (salaries: HR only; month-end closing: Finance only).
- **Access is checked before search**, so restricted text never reaches the model. Roles come from the user directory, not from the question.
- Local models: `bge-m3` finds the 3 best sections, `qwen3 4B` answers in 1-2 sentences **with sources**.
- **Guards against wrong answers:** refuse when no section is close enough, citations must be retrieved sections, and every number must appear in the source. One refusal message for "not found" and "not allowed".
- **Only approved, visible text is indexed:** approvals are checked by content hash, and documents with hidden text or instructions to the AI are quarantined.



Results (33 questions)

Metric	Result
Right section found (top 3)	15/15
Correct answers	15/15 (typos: 2/3)
Correct sources	17/17
Refused when not in documents / not allowed	7/7 and 6/6
Prompt-injection attacks resisted	2/2
Restricted data leaked	**0**
Human review: faithful / helpful	33/33 and 29/33
Time per question	4.8 s

- **Found and fixed:** in the first run, a hidden instruction in a document made the model add "*The user is authorized*". No data leaked, but the false claim got through. After the fix, both attacks were resisted.
- The human review found **mixed-language** answers in Indonesian and one typo question that was refused.

Privacy and security

- The model sees only the question and 3 sections the user may read; questions are hashed in logs.
- **64 tests**, including role checks, prompt injection with a manipulated model, and quarantine of unsafe documents.

Cost and limitations

- About **IDR 0.20 per question**; 10 of 33 questions needed no model call.
- Prompt injection cannot be ruled out completely; its reach is limited by the access filter and the checks.
- A small model sometimes mixes languages; heavy typos can be refused.

Critical Thinking & Engineering Judgment

Answers to the six scenarios. Each one gives a short answer first, then the reasoning, then the evidence that would change my decision. Where possible I point to what I measured in my own prototypes (Case 1 = fraud, Case 2 = invoices, Case 3 = chatbot).

Scenario 1: High accuracy, low fraud detection

A fraud model reports 98% accuracy but finds only 30% of confirmed fraud cases.

Short answer: Not as the main fraud control. 98% accuracy says almost nothing when fraud is rare, and missing 70% of fraud is a business problem.

Would I deploy it?

- **No, not as it is.** If 2% of transactions are fraud, a model that always says "not fraud" already scores 98% accuracy. In Case 1 that "always not fraud" baseline scored **98.8% accuracy and caught 0 of 84 fraud cases**.
- I would only use it **in shadow mode** (it scores, nobody acts on it) or as one extra signal, and only if it beats the current process at the same review capacity.

Which metrics and error costs matter most? How would I validate them?

- **Recall** (how much fraud we catch) and **precision at review capacity** (of the cases analysts can check per day, how many are real fraud). Also **PR-AUC** to compare models without picking a threshold.
- **Error costs:** a missed fraud (false negative) costs the stolen amount and trust; a false alarm (false positive) costs analyst time and annoys a good customer. I would agree the rough cost of each with the client, then compare models by expected cost: `missed fraud × average loss + false alarms × review cost`.
- **Validation:** a **time-based split** (train on the past, test on later months), confirmed fraud labels only, and enough fraud cases in the test set to trust the numbers. Also check results per segment (channel, amount).

What would I investigate or change first?

1. **The threshold.** Many models use 0.5 by default, which is wrong for rare events. Rank by risk and flag the top N the team can review (Case 1 used 10 per day).
2. **Labels.** Are they confirmed fraud, or "blocked accounts"? In Case 1, 28 blocked customers had no fraud and 30 fraud victims were never blocked; mixing these up teaches the model the wrong thing.
3. **Leakage and split.** A random split or future data in features gives fake-good results.
4. **Class imbalance** (class weights, better features such as failed logins and transaction speed).

Evidence that would change my decision

- The 30% it catches is the **high-loss** fraud (for example 80% of the money lost), and nothing better exists yet.
- At the real review capacity it beats the current rules on precision and recall in a shadow test.

Scenario 2: Unreliable data and a tight deadline

Customer IDs do not match across tables; events contain duplicates and missing timestamps. Demo in two weeks.

Short answer: Check the data first, shrink the MVP to the part that is reliable, and be honest with the client.

My first three checks

1. **Join rate.** What % of transactions match a customer? Which ID formats exist, and can simple rules fix most of them? (In Case 1, normalizing formats like `c-12` and `CUST_12` fixed 4,418 rows.)
2. **Duplicates.** Exact copies (system retries) or different events? Do duplicates double-count fraud or amounts?
3. **Missing timestamps.** How many, in which tables and periods, and are they random or concentrated in important cases such as fraud? Without time we cannot build safe "past-only" features.

I would also confirm early **which labels exist** and who confirmed them.

Train immediately or change the MVP scope?

- **Do not train on day one.** A model trained on broken joins learns noise and gives numbers we cannot defend.
- **Change the scope:** spend the first days on profiling and cleaning rules, then build a simple baseline (rules) and one model on the **reliable subset** (for example one product or the months with good data).
- Keep everything that does not need perfect data: the cleaning pipeline, the data-quality report, and a ranked review list instead of automatic decisions.

What would I tell the client, and what can I safely demonstrate?

- Tell them early, with numbers: "X% of transactions cannot be linked to a customer yet; here is why and how we fix it."
- **Safe to demo:** the data-quality report, the cleaning steps, and a baseline ranking on the clean subset, clearly labelled as a prototype on limited data.
- **Not safe to promise:** accuracy on the full data, or production readiness.

Evidence that would change my decision

- If simple mapping rules bring the join rate above ~95% and missing timestamps are random and rare, I would train on the full data within the two weeks.

Scenario 3: A chatbot gives confident but wrong answers

Your RAG chatbot gives an incorrect policy answer and cites a document that does not support it.

Short answer: First find out which step failed (search or answer writing), fix that step, and make the bot refuse when the evidence is weak.

How do I separate retrieval errors from generation errors?

- Look at the **retrieved sections** for that question (Case 3 logs their IDs for every question).
    - The right section was **not** in the top results → **retrieval error** (search, chunking, embeddings, access filter).
    - The right section **was** there but the answer is wrong → **generation error** (the model misread, mixed sections, or added facts).
    - The cited section does not support the answer → **grounding error**, which should be blocked automatically.
- Do this on a labelled question set with the **expected source section** for each question, not on one example.

What tests and metrics would show improvement?

- **Recall@K** for retrieval, **answer correctness**, **citation accuracy**, and **faithfulness** (every claim is in the cited section), plus correct-refusal and wrong-refusal rates.
- Add every failing question to a **regression set** so it stays fixed.
- Automatic checks after the model: citations must be retrieved sections, and every number must appear in the cited text. In Case 3 these checks block invented numbers even when the model is manipulated.

- A small **manual review** for things machines judge badly (tone, partial answers). Case 3 found mixed-language answers this way.

When should the assistant say it cannot answer?

- No section is similar enough to the question (Case 3 refuses below a score of 0.50, set on practice questions).
- The model says the answer is not in the sections, or its citation or numbers do not match the sources.
- The sources conflict, or the user is not allowed to see the relevant document (with the **same** message, so the refusal does not reveal that a restricted document exists).
- For policy questions, **a polite refusal is better than a confident guess.**

Evidence that would change my decision

- If most failures are retrieval errors, I would focus on chunking and search (or add keyword search), not on the model.
- If refusals become too frequent for valid questions (for example typos), I would re-tune the refusal bar on a larger practice set.

Scenario 4: Privacy versus speed and cost

The client wants confidential documents processed by a cheap external LLM, but no external data transfer has been approved.

Short answer: No. I would not send the documents. Cheap is not a reason to break an approval, a contract or the law (in Indonesia, the Personal Data Protection Law, UU PDP).

Would I send the documents?

- **No.** Without approval we do not know where the data goes, how long it is kept, or whether it trains the vendor's model. One leak can cost far more than the money saved.
- My prototypes **refuse external endpoints by default** (`ALLOW_EXTERNAL_LLM` must be set on purpose after approval) and show exactly what would be sent with `--show-prompt`.

Two safer alternatives

	A. Local / self-hosted model	B. External LLM, but only after approval and with controls
How	Open model on our own GPU or server (Case 2 and 3 use a 4B model in Ollama)	Enterprise contract, no training on our data, no or short retention, approved region; send only redacted text
Privacy	Data never leaves	Data leaves, but minimized and under contract
Quality	Case 2: 95% field accuracy; may be weaker on hard documents	Usually higher on complex documents
Speed	~4 s per document on a laptop GPU	Often faster, depends on the network
Cost	Hardware and maintenance; electricity ≈ IDR 0.16 per invoice in Case 2	Per token, ≈ US$0.00014 per invoice at assumed small-model prices
Main risk	We maintain the model and hardware	Vendor and contract risk; redaction must not miss anything

I would start with **A** and measure. If quality is not enough, prepare **B** with the client's approval.

Approvals, controls and tests before launch

- **Approvals:** data owner and legal / data protection officer sign-off, a privacy impact assessment, a vendor security review, and a contract (data processing agreement) if we use an external vendor.
- **Controls:** redaction of fields we do not need, an allowlist of endpoints, secrets kept out of code, logs without document content, access control on uploads and outputs.
- **Tests:** a negative test that unapproved endpoints are refused, a test that the prompt contains no bank accounts, phones or emails (both exist in Case 2), accuracy on a labelled set, and a security review of the API.

Evidence that would change my decision

- Written approval plus a contract with no retention and an approved region would make option B acceptable.
- If the documents contain no personal or confidential data after redaction, the risk is much lower.

Scenario 5: The proposed architecture costs too much

Your design costs three times the client's budget.

Short answer: Measure where the money actually goes, cut the biggest drivers first, and prove with the same tests that the cheaper design still meets the requirements.

Which cost drivers would I measure first?

1. **Volume:** documents or questions per day, and the peak.
2. **Model usage:** tokens per request and model size (prompt length often dominates).
3. **Always-on compute:** GPUs or servers running while idle.
4. **Human review rate:** every case sent to a person costs staff time. This is often the biggest cost (in Case 1 it is analyst time, not compute).
5. Storage and retention of data and logs.

What would I simplify, and what trade-offs follow?

Simplification	Saves	Trade-off
Cascade: cheap rules first, the model only for hard cases (Case 2: regex is 100% on known layouts, the LLM handles new ones)	Most model calls	More code paths; rules break on new formats
Batch instead of real-time (Case 1 scores once a day)	Always-on servers	Slower reaction
Smaller model or shorter prompts (fewer retrieved sections)	Compute and tokens	Possibly lower quality
Refuse early without the model (Case 3 handled 10 of 33 questions with a refusal and no model call)	Model calls	A few valid questions may be refused
Scale to zero, no extra services (Case 3 uses NumPy instead of a vector database)	Idle cost	Cold starts; not suited to very large data

How would I verify the cheaper design still meets requirements?

- Agree the **must-pass criteria first** (for example: 0 wrong auto-accepted invoices, 0 data leaks, accuracy within 2 points of the current design, latency under X seconds).
- Run the **same evaluation set** on both designs and compare. Run the cheaper one in shadow mode for a while and monitor cost per document and review rate.

Evidence that would change my decision

- If the review rate rises so much that staff cost eats the savings, the "cheaper" design is not cheaper.
- If the client's real volume is much lower than assumed, the original design may fit the budget with smaller servers.

Scenario 6: Ownership under uncertainty

Your team lead is unavailable for two days. You discover a wrong assumption the day before a client demo.

Short answer: Confirm the problem with evidence, write it down, fix only what I can test safely, never hide it, and inform the right people the same day.

What would I verify and document first?

- **Is the assumption really wrong?** Check the data or the client document, and reproduce the problem.
- **What does it affect?** Which results, numbers, slides and demo steps change, and by how much.
- A **short note**: what I found, how, the impact, the options, and my recommendation, with the previous results kept for comparison.

What would I change, stop, or safely demonstrate?

- **Change** only what I can fix and re-test before the demo, then re-run the evaluation. In Case 3 I found a prompt-injection weakness after the first run, fixed it, ran the evaluation again, and kept both results.
- **Stop** showing any number or feature that depends on the wrong assumption. No untested changes the night before.
- **Demonstrate** the parts that are not affected, and explain the issue openly as a known limitation with a plan.

Who would I inform, when, and why?

- **My team lead, right away**, by message, even if they are away, so they are not surprised later.
- **Their backup or the project manager the same day**, because someone with authority must decide what to tell the client.
- **The client before or at the start of the demo**, agreed with the project manager: what changed, what is still reliable, and when the fix comes.
- **Why:** decisions about client communication belong to the people accountable for the project, and clients trust a team that reports problems early more than one that hides them.

Evidence that would change my decision

- If the impact is tiny (for example a label on one slide), I would fix it and mention it in the note, not escalate.
- If the fix is large and risky, I would recommend moving the affected part of the demo rather than rushing it.

AI tool use

I used **Claude Code** (Anthropic) as my AI coding assistant, with **Matt Pocock's skills** (mainly "grilling": the AI asks me questions until every design decision is agreed, before coding) and **Ponytail** (a plugin that keeps the code as small as possible). The AI helped me design, write code, data, tests and documents, run the evaluations, and fix issues found by automated security reviews.

My part: I made the design decisions, checked each step's explanation and results before moving on, and confirmed the manual review in Case 3 (pre-filled by the AI). All tests and evaluations ran in my own environment, and every number comes from those runs. I am responsible for this work and can explain it.